

VS Code + Codex Full Project Implementation Guide

Hotel Booking Cancellation Prediction Using Machine Learning

Project item	Details
Dataset	Hotel Booking Demand
Original size	119,390 records x 32 columns
Target	is_canceled (0 = not cancelled, 1 = cancelled)
Task	Supervised binary classification
Current status	PE1: EDA, preprocessing and feature engineering completed
Next work	Model development, validation, tuning, final model, application, testing and technical-report evidence

Purpose of this guide

Use this PDF inside your VS Code project together with the official IT3051 Mini Project Assignment PDF, hotel_bookings.csv and your existing PE1 notebook. Give Codex the master instruction in this document. Codex should continue the project from your existing work, execute the experiments, and generate verified result files that can later be used to finish the Technical Report.

1. Files to Put in the VS Code Project

Create a clean project folder and keep your original PE1 notebook unchanged as evidence of the work already completed.

```
hotel-booking-cancellation/ |-- data/ | `-- hotel_bookings.csv |-- notebooks/ | `-- PE1_analysis.ipynb |--
models/ |-- outputs/ | |-- figures/ | `-- results/ |-- app/ | `-- app.py |--
IT3051_FDM_Mini_Project_Assignment_2026.pdf |-- requirements.txt |-- README.md `-- .gitignore
```

Also include the existing PE1 notebook. The final project should continue from it rather than silently replacing the methodology.

2. Project Facts Codex Must Preserve

Item	Current project value
Project	Hotel Booking Cancellation Prediction Using Machine Learning
Dataset	Hotel Booking Demand
Original dataset	119,390 rows, 32 columns
Target	is_canceled
Target meaning	0 = not cancelled; 1 = cancelled
Original target distribution	75,166 not cancelled (62.96%); 44,224 cancelled (37.04%)
Task	Supervised binary classification
Duplicates found	31,994
Rows after duplicate removal	87,396
Train/test split	80/20, random_state=42, stratify=y
Training rows	69,916
Testing rows	17,480
Processed features	254

3. Existing PE1 Work

The following work has already been completed and should be inspected before any new changes are made.

- Problem and dataset understanding
- Exploratory Data Analysis
- Missing-value analysis
- Duplicate analysis
- Outlier investigation
- Target distribution and class imbalance
- Relationship and correlation analysis
- Data preprocessing
- Feature engineering
- Data-leakage prevention
- Train/test preparation

Missing values

Column	Missing values	Current treatment
children	4	Median
country	488	'Unknown'
agent	16,340	0
company	112,593	0

Feature engineering

Feature	Definition
total_nights	stays_in_weekend_nights + stays_in_week_nights
is_long_stay	1 if total_nights > 7, otherwise 0
is_weekend_stay	1 if weekend nights > 0, otherwise 0
is_high_lead_time	1 if lead_time > 60, otherwise 0

Leakage variables already removed: reservation_status and reservation_status_date. Numerical preprocessing uses median imputation and StandardScaler. Categorical preprocessing uses most-frequent imputation and OneHotEncoder(handle_unknown='ignore').

Do not claim that outliers were removed, SMOTE was used, or total_guests was engineered unless those steps are actually added, executed and justified later.

4. Audit Before Model Development

Ask Codex to inspect the existing notebook and dataset before Stage 6. It should check the following points and explain any proposed correction before changing your PE1 methodology.

- Whether removing all 31,994 exact duplicate rows is methodologically justified
- Missing-value treatment
- Whether agent and company should be treated as continuous numerical features
- Outlier handling
- Categorical encoding and numerical scaling
- Leakage variables and prediction-time availability
- Train/test split and preprocessing leakage
- Feature engineering and feature selection
- Class imbalance

Important: Codex should not silently change PE1. It should state the issue, why it matters, the recommended change, and whether the change affects previous PE1 results.

5. Stage 6 - Model Development

Implement at least four suitable classification algorithms using the same training/testing data and preprocessing logic for a fair comparison.

Model	Reason
Logistic Regression	Interpretable linear baseline for binary classification.
Decision Tree	Captures non-linear rules and interactions.
Random Forest	Ensemble of trees that can improve robustness.
Gradient Boosting	Sequential ensemble capable of modelling complex non-linear patterns.

For every model calculate Accuracy, Precision, Recall, F1-score and ROC-AUC. Also create a confusion matrix, ROC curve and classification report. Save the baseline results to a pandas DataFrame and CSV.

Do not choose the final model only from accuracy. Because the target is moderately imbalanced, class-specific metrics are important.

6. Validation Strategy

Use a suitable cross-validation strategy on the training data, preferably StratifiedKFold where appropriate. Keep the test set untouched until final evaluation. Document the number of folds, shuffle setting, random state and scoring metrics. Any preprocessing that learns from data must be fitted inside the training process/pipeline so test-set information cannot leak into training.

7. Stage 7 - Hyperparameter Tuning

After baseline comparison, tune promising models using GridSearchCV or RandomizedSearchCV depending on computational cost. Record baseline performance, tuned performance, best parameters, cross-validation score and test performance.

Model	Possible parameters
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features
Gradient Boosting	n_estimators, learning_rate, max_depth, subsample
Decision Tree	max_depth, min_samples_split, min_samples_leaf, criterion
Logistic Regression	C, penalty, compatible solver

Only use compatible parameter combinations and report the values that were actually tested.

8. Final Model Selection and Interpretation

Select the final model using actual experimental evidence. Consider Precision, Recall, F1-score, ROC-AUC, cross-validation stability, generalization, overfitting, interpretability and computational requirements. Do not automatically select the highest-accuracy model.

For a tree-based final model, map and report feature_importances_. For Logistic Regression, use coefficients. Map transformed one-hot feature names correctly and create a top-feature chart. Describe features as associated with prediction or important to the model; do not make causal claims.

9. Save a Complete Inference Pipeline

Create a complete inference pipeline containing the required preprocessing and final trained model. Save it with joblib, for example models/hotel_cancellation_pipeline.joblib. Test loading it in a fresh script/process and passing a raw booking record to it.

10. Backend and Frontend

Backend workflow: raw booking input -> validation -> feature engineering -> exact saved preprocessing -> final model -> prediction/probability -> clear result. Never request is_canceled, reservation_status or reservation_status_date as prediction inputs.

For the frontend, Streamlit is a practical choice. Use clear sections such as Booking Information, Guest Information, Stay Information, Customer History and Reservation Information. Use select boxes for categorical fields and number inputs for numerical fields. Calculate total_nights, is_long_stay, is_weekend_stay and is_high_lead_time internally rather than asking the user to enter them.

The result can display 'Likely to Cancel' or 'Not Likely to Cancel'. If the selected model provides a meaningful probability, display it as a model estimate rather than a guaranteed outcome.

11. System Testing

Create and record end-to-end tests for the final application.

Test case	Expected behaviour
Valid normal booking	Prediction is returned.
Missing required value	Clear validation message.
Invalid numerical input	Input rejected or validation shown.
Unseen categorical value	Pipeline handles it safely.
Saved pipeline loading	Model/pipeline loads successfully.
Feature engineering	Derived features are calculated correctly.
Prediction output	Valid class and, where appropriate, probability returned.

12. Required Output Files

```
outputs/ |-- figures/ | |-- confusion_matrix_logistic_regression.png | |--  
confusion_matrix_decision_tree.png | |-- confusion_matrix_random_forest.png | |--  
confusion_matrix_gradient_boosting.png | |-- roc_comparison.png | |-- feature_importance.png |-- results/  
|-- baseline_model_results.csv |-- tuning_results.csv |-- final_model_metrics.json |--  
system_test_results.csv models/ |-- hotel_cancellation_pipeline.joblib
```

13. Technical Report Evidence File

After all experiments are executed, ask Codex to create `TECHNICAL_REPORT_RESULTS.md` containing only verified results. This file should contain the dataset summary, EDA summary, preprocessing, feature engineering, four algorithms and rationale, baseline results, validation strategy, tuning, tuned results, final model selection, final metrics, feature importance, system architecture, backend/frontend implementation, testing, limitations and future improvements.

Do not allow Codex to invent missing metrics, figures, hyperparameters or system-test outcomes. Any result in the final report must come from executed code or a clearly documented design decision.

14. README and Requirements

Create `README.md` with project title, problem, dataset, project structure, installation, Python version, dependency installation, training instructions and application launch instructions. Create `requirements.txt` containing only packages actually used, such as pandas, numpy, scikit-learn, matplotlib, seaborn, joblib and streamlit where applicable.

Typical application command:

```
streamlit run app/app.py
```

15. Master Prompt to Paste into Codex

Paste the following after placing the official assignment PDF, dataset and PE1 notebook in the VS Code project.

I am completing my university mini project for IT3051 - Fundamentals of Data Mining.
Project: Hotel Booking Cancellation Prediction Using Machine Learning.

FIRST:

1. Read the IT3051 Mini Project Assignment PDF in this repository completely.
2. Inspect my existing PE1 Jupyter notebook and dataset before changing anything.
3. Do not invent results. All metrics, graphs, hyperparameters, feature importance values and final-model decisions must come from executed code.
4. Do not unnecessarily redo or delete my completed PE1 work.

PROJECT FACTS:

- Dataset: Hotel Booking Demand
- Original dataset: 119,390 rows x 32 columns
- Target: is_canceled
- 0 = not cancelled; 1 = cancelled
- Task: supervised binary classification
- Original target: 75,166 not cancelled (62.96%), 44,224 cancelled (37.04%)
- Missing values: children 4, country 488, agent 16,340, company 112,593
- Duplicates identified: 31,994
- Rows after current duplicate removal: 87,396
- Current missing treatment: children median; country 'Unknown'; agent 0; company 0
- Current categorical preprocessing: OneHotEncoder(handle_unknown='ignore')
- Current numerical preprocessing: median imputation + StandardScaler
- Engineered features: total_nights, is_long_stay, is_weekend_stay, is_high_lead_time
- Leakage fields removed: reservation_status, reservation_status_date
- Split: 80/20, random_state=42, stratify=y
- Train rows: 69,916; test rows: 17,480
- Processed features: 254
- Do not claim outliers were removed, SMOTE was used, or total_guests was created unless actually implemented and justified.

AUDIT:

Before modelling, audit duplicate handling, missing-value handling, agent/company treatment, outliers, encoding, scaling, leakage, prediction-time availability, train/test split, feature engineering, feature selection and class imbalance. Do not silently change PE1. Explain the issue, why it matters, recommended change and whether it affects previous PE1 results.

STAGE 6:

Implement Logistic Regression, Decision Tree, Random Forest and Gradient Boosting using the same data/preprocessing logic. For every model calculate Accuracy, Precision, Recall, F1-score and ROC-AUC; create confusion matrix, ROC curve and classification report. Save baseline_model_results.csv.

VALIDATION:

Use appropriate cross-validation on training data, preferably StratifiedKFold where appropriate. Keep test data untouched until final evaluation. Document folds, shuffle, random_state and scoring. Prevent preprocessing leakage.

STAGE 7:

Tune promising models with GridSearchCV or RandomizedSearchCV. Record baseline vs tuned results, best parameters, CV score and test performance. Save tuning_results.csv. Do not choose parameters without actually testing them.

FINAL MODEL:

Select from actual evidence using precision, recall, F1, ROC-AUC, CV stability, generalization, overfitting, interpretability and computational requirements. Do not select only by accuracy.

INTERPRETATION:

For the selected model, correctly map transformed feature names and produce feature importance or coefficients. Create feature_importance.png. Do not make causal claims.

SAVE PIPELINE:

Save a complete raw-input-to-prediction pipeline with joblib as models/hotel_cancellation_pipeline.joblib. Test loading it in a fresh script/process.

BACKEND:

Create reusable prediction logic that validates raw input, creates engineered features, applies exactly the training preprocessing, predicts, optionally returns probability, and handles invalid input. Never request is_canceled, reservation_status or reservation_status_date.

FRONTEND:

Use Streamlit unless there is a strong reason not to. Create app/app.py with clear booking input sections and a Predict Cancellation Risk button. Automatically calculate engineered features. Display Likely to

Cancel / Not Likely to Cancel and, if valid, a model probability.

TESTING:

Test valid booking, missing required value, invalid numeric input, unseen category, pipeline loading, feature engineering and prediction output. Save system_test_results.csv.

OUTPUTS:

Save confusion matrices for all four models, roc_comparison.png, feature_importance.png, baseline_model_results.csv, tuning_results.csv, final_model_metrics.json and system_test_results.csv.

REPORT SUPPORT:

Create TECHNICAL_REPORT_RESULTS.md containing only verified executed results: dataset, EDA, preprocessing, feature engineering, algorithms, baseline results, validation, tuning, tuned results, final model, metrics, interpretation, architecture, backend, frontend, testing, limitations and future improvements.

README:

Create README.md with project structure, installation, dependencies, training command and Streamlit launch command.

CODE QUALITY:

Use clear names, comments, functions, reproducible random states and modular code. Keep EDA, preprocessing, training, evaluation, tuning, inference and frontend clearly separated. Do not delete my original PE1 notebook. Keep the implementation understandable enough for every group member to explain in an individual viva.

AT THE END GIVE ME:

1. Every file created/modified.
2. Exact dependency-install command.
3. Exact model-training command.
4. Exact frontend launch command.
5. Final model comparison.
6. Best hyperparameters.
7. Selected final model and evidence-based reason.
8. Remaining issues/limitations.
9. A checklist mapping completed work to the official IT3051 assignment requirements.

16. What to Upload Back to ChatGPT

After Codex finishes and the code has been executed successfully, upload the following files here so the final Technical Report can be completed with real results.

- TECHNICAL_REPORT_RESULTS.md
- baseline_model_results.csv
- tuning_results.csv
- final_model_metrics.json
- system_test_results.csv
- model_experiments.ipynb or the final modelling notebook
- A PDF containing the final outputs/screenshots if convenient

17. Complete Workflow

Assignment PDF + dataset + PE1 notebook -> Codex audit -> four baseline models -> evaluation -> cross-validation -> tuning -> final model -> feature interpretation -> saved pipeline -> backend -> Streamlit frontend -> system testing -> verified result files -> final Technical Report -> PE2 viva -> final presentation/demo.

The key rule throughout the project is simple: do not write a result in the report unless the code actually produced it.